# Report

**Interactive Symbol Table Compiler**

C++ Source Code Analysis & Symbol Extraction Tool

Name: Sudharsan S
Roll No: CH.SC.U4CYS23045

Amrita Vishwa Vidyapeetham
Department of Computer Science & Engineering
April 2026

## 1. Introduction

A symbol table is a fundamental data structure used by compilers and interpreters to store information about identifiers (variables, functions, parameters) found in source code. It maps each identifier to its associated attributes — type, scope, value, and line number — enabling semantic analysis, type checking, and code generation.

This report documents the design and implementation of an Interactive Symbol Table Compiler — a browser-based tool that accepts real C++ source code and produces a structured symbol table in real time. The tool is built entirely in HTML, CSS, and JavaScript, requiring no server-side compilation or external libraries.

The system is capable of parsing a wide range of C++ constructs including preprocessor directives, global and local variable declarations, static variables, function definitions, and function parameters. Each symbol is classified by its type, inferred scope, initial value, and source line number.

## 2. Objectives

The primary objectives of this project are:

- Accept arbitrary C++ source code as input through a browser-based editor
- Tokenise the source into a structured token stream (TYPE, ID, LIT, ASSIGN, SEMI, etc.)
- Parse the token stream to identify and classify all symbols
- Detect variable declarations with full type resolution including modifiers
- Infer variable scope: global, local (within functions), static, or parameter
- Identify and log function definitions along with their return types

- Display a sortable, filterable symbol table with type badges and scope indicators
- Handle C++ specific constructs: #include, using namespace, static, const, block comments
- Provide test cases for common scenarios including type mismatches and update cases

## 3. System Architecture

The tool is implemented as a single-page HTML application divided into four logical panes:

| Pane | Title | Function |
|---|---|---|
| 1 | Source Editor | Code input with line numbers, syntax-aware textarea |
| 2 | Compiler Output | Line-by-line parse log with OK / WARN / INFO tags |
| 3 | Symbol Table | Extracted symbols with sort and scope filter controls |
| 4 | Token Stream | Visual token chips for each line of source code |

The processing pipeline operates in three stages:

1. Tokenisation — The source is scanned character by character to produce a token stream. Tokens are classified as TYPE, ID, LIT, MOD (modifier), KW (keyword), ASSIGN, SEMI, LPAREN, RPAREN, LBRACE, RBRACE, STAR, AMP, SCOPE (::), COMMA, and UNK.
2. Parsing — The token stream is analysed line by line with a context-aware parser that tracks brace depth, function stack, and class stack to determine the correct scope for each symbol.
3. Rendering — Parsed symbols are displayed in a structured HTML table with colour-coded type badges, scope indicators, and kind labels (VAR or PARAM).

## 4. Tokeniser Design

The tokeniser is a hand-written scanner that processes each line of source code. Block comments (/* ... */) are stripped in a pre-processing pass using a regular expression. Single-line comments (//) are stripped per line during tokenisation.

The following token types are produced:

| Token Type | Examples | Description |
|---|---|---|
| TYPE | int, float, double, char, string, bool, void | Primitive C++ type keywords |
| MOD | static, const, unsigned, signed | Type modifier keywords |
| KW | if, while, return, cout, class, namespace | C++ reserved keywords (non-type) |

| Token Type | Examples | Description |
|---|---|---|
| ID | globalVar, demoFunction, localVar | User-defined identifiers |
| LIT | 100, 3.14, 'A', "hello", true, false | Literal values of any type |
| ASSIGN | = | Assignment operator |
| SEMI | ; | Statement terminator |
| LPAREN | ( | Left parenthesis |
| RPAREN | ) | Right parenthesis |
| LBRACE | { | Block open |
| RBRACE | } | Block close |
| SCOPE | :: | Scope resolution operator |
| STAR | * | Pointer declarator |
| AMP | & | Reference declarator |
| UNK | ++, <<, >> | Unrecognised / operator tokens |

# 5. Parser Design

The parser maintains three pieces of mutable state across lines to correctly infer scope:

- braceDepth — Tracks the current nesting depth of {} blocks
- funcStack — A stack of {name, returnType} entries, one per open function body
- classStack — A stack of {name, access} entries, one per open class/struct body

Each line is classified as one of the following and processed accordingly:

| Line Type | Detection Rule | Action |
|---|---|---|
| Preprocessor directive | Line starts with # | Log as INFO, skip symbol extraction |
| Using directive | First token is KW 'using' | Log as INFO, skip |
| Function definition | [mods] TYPE ID ( ... { | Push to funcStack, extract parameters |
| Variable declaration | [mods] TYPE ID [= value] ; | Add to symbol table with scope |
| Variable reassignment | ID = value ; | Update existing symbol, mark as updated |
| Other statement | Everything else | Log as INFO with line preview |

## 5.1 Scope Inference

The scope of each symbol is inferred from the parser state at the time the declaration is encountered:

| Detected Scope | Condition | Display |
|---|---|---|
| global | No open function or class body | public/global or private/global |
| local | Inside a function body, not static | local |
| static | Inside a function body with static modifier | static |
| param | Declared in a function's parameter list | param:functionName |
| public | Inside a class/struct body, access = public | public |
| private | Inside a class/struct body, access = private | private |

### 5.2 Type Resolution

Types are resolved by the buildType() function which collects modifier tokens (static, const, unsigned), the base type token (TYPE or user-defined ID), optional template parameters (e.g. vector<int>), and pointer/reference decorators (* and &). The resulting type string is stored verbatim in the symbol table, for example: const int, unsigned long, string, or int*.

## 6. Test Cases

Five test cases are provided within the tool, each demonstrating a distinct parsing scenario.

### Test Case 1 — Basic Declarations

**Input:**

```
int a = 5;
float b = 2.5;
```

**Expected Symbol Table:**

| Name | Type | Value | Scope | Kind | Line |
|---|---|---|---|---|---|
| a | int | 5 | public/global | VAR | 1 |
| b | float | 2.5 | public/global | VAR | 2 |

### Test Case 2 — Update / Redeclaration

**Input:**

```
int a = 5;
int a = 10;
```

*The second declaration updates the existing entry for 'a' rather than creating a duplicate. The symbol is marked as updated.*

**Expected Symbol Table:**

| Name | Type | Value | Updated | Line |
|---|---|---|---|---|
| a | int | 10 | Yes | 2 |

## Test Case 3 — Type Mismatch Error

**Input:**

```
string x = 4;
```

*Assigning a numeric literal (4) to a string variable is a type mismatch. The parser flags this with an ERR log entry and does not add the symbol to the table.*

**Expected Output Log:**

| Tag | Line | Message |
|---|---|---|
| ERR | 1 | Type mismatch (cannot assign 4 to string) |

## Test Case 4 — Mixed Valid and Invalid

**Input:**

```
int a = 5;
char c = "A";
string name = "Sudhan";
```

*Line 2 is a type mismatch (char assigned a string literal). Lines 1 and 3 are valid and produce symbol entries.*

| Name | Type | Value | Scope | Line |
|---|---|---|---|---|
| a | int | 5 | public/global | 1 |
| name | string | "Sudhan" | public/global | 3 |

## Test Case 5 — C++ Scope Demonstration

**Input:**

```
#include <iostream>
using namespace std;

int globalVar = 100;

void demoFunction() {
    static int staticVar = 0;
    int localVar = 10;
    staticVar++;
}

int main() {
    demoFunction();
    return 0;
}
```

**Expected Symbol Table:**

| Name | Type | Value | Scope | Kind | Line |
|---|---|---|---|---|---|
| globalVar | int | 100 | public/global | VAR | 4 |
| staticVar | int | 0 | static | VAR | 7 |
| localVar | int | 10 | local | VAR | 8 |

# 7. Key Features

## 7.1 Sort Controls

The symbol table supports four sort modes: by insertion Order, alphabetically by Name (A→Z), by Type, and by Scope. Sorting is not applied live — the user must select a mode and then click the Sort button, preventing accidental reordering during active editing.

## 7.2 Scope Selector

A dropdown in the editor toolbar allows the user to set the base scope (Public or Private) for global-level declarations. This setting is applied at compile time and is reflected in the Scope column of the symbol table.

## 7.3 Token Stream Viewer

Every line of the source is independently tokenised and its tokens displayed as colour-coded chips in the Token Stream pane. This provides a visual representation of the lexical analysis phase and helps diagnose parsing errors.

### 7.4 Compiler Output Log

The output log records every parsing event with a severity tag:

| Tag | Meaning |
|---|---|
| OK | Symbol successfully added to the symbol table |
| WARN | Existing symbol updated with a new value |
| ERR | Parse or type error — symbol not added |
| INFO | Informational event (preprocessor, function definition, statement) |

## 8. Limitations & Future Work

The current implementation has several limitations that could be addressed in future iterations:

- Multi-line declarations (e.g. int a = foo() + bar();) are not fully supported — the parser is line-oriented
- Macro definitions (#define) are skipped without symbol extraction
- Template class declarations (template<typename T>) are parsed but template parameters are not entered into the symbol table
- Enum, union, and typedef declarations are not currently recognised
- Operator overloads and lambda expressions are treated as unrecognised statements
- No semantic type inference for auto variables

Planned enhancements include multi-line statement joining, a proper AST construction phase, support for enum and typedef, and an exportable symbol table in CSV or JSON format.

## 9. Conclusion

The Interactive Symbol Table Compiler successfully demonstrates the core principles of lexical analysis and symbol table construction as applied to C++ source code. The tool operates entirely in the browser, provides real-time feedback through a structured output log, and presents results in a clean, sortable symbol table.

By handling real C++ constructs — including preprocessor directives, scope modifiers, function definitions, static variables, and function parameters — the tool bridges the gap between simple assignment-based parsers and genuine compiler front-end behaviour. It serves as a practical educational tool for students studying compiler design and programming language theory.